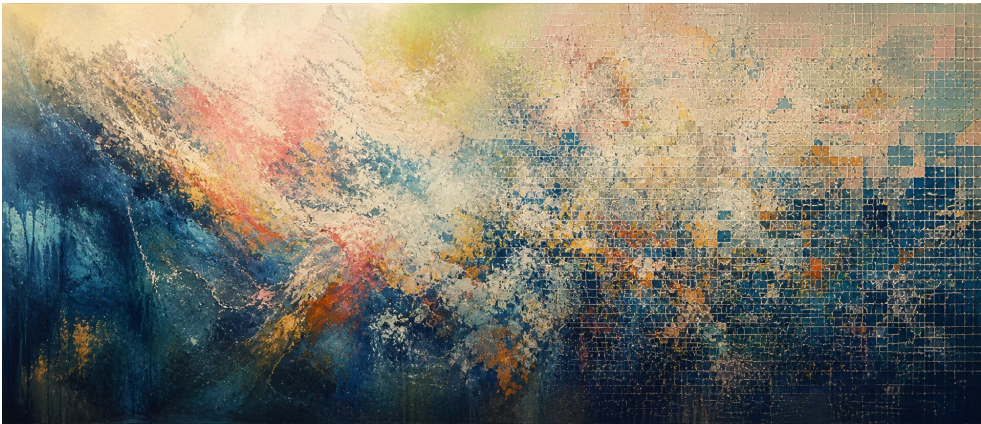


On the Non-Compositionality of Intelligence

SIDDHANT SAXENA · @sidgraph · CLAUDE OPUS 4.7

The University is just the way in which all that he has already seen is organized. When they are seen and when their coordination is understood, the University has been seen. (Gilbert Ryle, The Concept of Mind, 1949)



A quiet structural claim runs underneath much of what we say about artificial intelligence: that intelligence composes. Take a capable language model, attach an interpreter, a retriever, a filesystem, and an orchestration loop, and the result is intelligent in the same sense the model alone might be, only more so. The components are well-behaved, the composition well-engineered, the benchmarks reward the assemblage. We point at the assemblage and call it intelligent.

This essay argues that intelligence is **non-compositional under externalization**: the property we name when we say “intelligent” is not preserved when its constitutive operations are distributed across components outside the system whose intelligence we are claiming. A model that cannot compute without delegating is not a computing system; it is an interface to one. A model that cannot reason without delegating the reasoning is not a reasoning model; it is a router for one. This is not a claim about whether such systems are useful: they are extremely useful. It is a claim about whether the words we use to describe them refer to anything specific. Most of the time, they don’t.

The asymmetry shows up immediately. Ask a frontier model “what is 847×392 ?” and either it works through long multiplication in its scratchpad and arrives at 332,024 on its own, or it emits a tool call, an external interpreter

returns 332,024, and the model relays the answer. From the outside both responses look like correct arithmetic. They are not the same kind of event. The first is computation. The second is delegation. Treating them as equivalent (as we routinely do when we report a model’s “math accuracy”) is the move this essay refuses.

The compositionality assumption

Some properties compose: mass, monetary value, deterministic correctness when each component is provably correct. Others don’t. Formally, a property P of systems is *compositional* under a composition operator $\odot$ if $P(A) \wedge P(B) \Rightarrow P(A \odot B)$. The interesting question for intelligence is which $\odot$ we have in mind. Function composition $f \circ g$ preserves type-correctness; so does deterministic, infinite-precision arithmetic. Approximate, finite-precision, batch-fused floating-point arithmetic does not: associativity fails by $O(\epsilon \cdot n)$ in the worst case, where ϵ is unit roundoff and n is the reduction width.

Thinking Machines Lab’s analysis (He, Sep 2025) makes the structural case cleanly. The popular story is that GPU floating-point non-associativity plus thread concurrency produces nondeterminism; He shows the actual culprit is **batch invariance failure**. Matmul, RMSNorm, and attention reduction kernels change numerical output for a given sample as a function of the surrounding batch, and batch composition shifts with concurrent load: *“Although the inference server itself can be claimed to be ‘deterministic,’ the story is different for an individual user. From the perspective of an individual user, the other concurrent users are effectively nondeterministic”* (He, 2025). Each component is run-to-run deterministic; the user sees nondeterminism. The published `batch_invariant_ops` library swaps RMSNorm, matmul, and attention via `torch.Library` and integrates with vLLM; one thousand temperature-zero runs collapse to a single bit-identical output, at a roughly 60% throughput cost. The earlier nondeterminism was real and located nowhere in particular.

The same seam reappears inside the training stack. The FlashRL group documented in 2025 that the trainer (HuggingFace forward pass) and the inference engine (vLLM) produce *different log-probabilities for the same sequence even with identical weights*, because their kernels differ — and quantization (FP8, INT8) widens the gap further. Concretely: let $\pi_{\text{train}}(\cdot \mid s)$ and $\pi_{\text{roll}}(\cdot \mid s)$ denote the trainer- and rollout-engine token distributions at state s . Even with identical parameters θ , the kernel differences yield $\pi_{\text{train}} \neq \pi_{\text{roll}}$, sometimes catastrophically: *“for certain tokens, the two policies even yield contradictory predictions — $\pi_{\text{vllm}}(a, \theta) = 1$ and $\pi_{\text{fsdp}}(a, \theta) = 0$, which breaks the on-policy assumption and secretly makes the RL training become off-policy”* (Yao et al., 2025). The stan-

dard on-policy PPO/GRPO surrogate is then biased; the only correction is a token-level truncated importance weight,

$$\mathcal{L}_{\text{TIS}}(\theta) = \mathbb{E}_{\tau \sim \pi_{\text{roll}}} \sum_t \min\left(\bar{\rho}, \frac{\pi_{\theta}(a_t|s_t)}{\pi_{\text{roll}}(a_t|s_t)}\right) \cdot \hat{A}_t \cdot \log \pi_{\theta}(a_t | s_t), \quad (1)$$

with clip $\bar{\rho} \in [2, 5]$. ROLL Flash (Oct 2025) systematizes this family — TIS, IcePop, MIS, WTRS — as token-level and sequence-level corrections for the same underlying defect. The “efficient RL framework” is therefore implicitly off-policy; without (1) the gradient estimator silently corrupts. Each component is correct against its own specification. The bug exists only in the seam.

Deterministic components can compose into a nondeterministic system, because “deterministic” is not preserved under composition.

Intelligence fails to compose for analogous reasons. A language model is an excellent next-token predictor; an interpreter is a correct evaluator; a vector store is a faithful retriever. We compose them and call the result intelligent, but “intelligent” is not a property that composes the way we are asking it to. The understanding-and-solving is the assemblage’s behavior; crediting it to the model is the same category error as crediting determinism to a kernel that lacks batch invariance.

This inverts Ryle’s foreign visitor. The visitor commits a category mistake by treating the University as another entity alongside the colleges that constitute it. We commit the mirror error: shown the model, the tools, the retrieval layer, and the orchestration loop, we say “the model is intelligent.” Both are failures of categorization with the same origin: an intuition that the property we care about lives somewhere it does not.

When you say “the model can do X,” the question is what the model is. If it is the network weights, doing X means the weights compute X. If it includes the interpreter, the retrieval layer, the filesystem, and the agent loop, “the model” is shorthand for an engineering organization plus a deployment stack. Both senses are coherent in isolation; the problem is sliding between them. We report benchmarks under sense two and draw conclusions about capabilities under sense one. The agent’s contribution — selecting the tool, marshaling arguments, parsing results — is what a competent operator with a function-calling API does. We would not say the operator can do the math. We say the model can.

Four properties that fail to compose

Verification. A tool call is auditable; a native computation is opaque. Auditability composes; competence does not. A model that calls an interpreter

correctly tells you nothing about whether the model could compute on its own.

Graceful degradation. A native system degrades smoothly; a delegating agent collapses sharply. By May 2026 the within-model gaps are smaller than at the start of the year, but they have not closed. On Humanity’s Last Exam (open-domain, contamination-resistant), GPT-5.5 scores 41.4% bare and 52.2% with tools; Opus 4.7 scores 46.9% / 54.7%; Gemini 3.1 Pro scores 44.4% / 51.4%; Mythos Preview, Anthropic’s withheld strongest model, gains 7.9 points (56.8% → 64.7%) when its full assemblage is brought online. CharXiv: Opus 4.7 jumps 82.1% → 91.0% with a single image-cropping tool. Anthropic’s own framing flags the no-tools number as isolating “the vision capability itself.” ARC-AGI-2: adding the open-source Poetiq harness raises Gemini 3 Pro from 31% to 54% at a 37× cost increase (\$0.81 → \$30.57/task). As of May 2026, no published frontier model scores higher without its tools than with them.

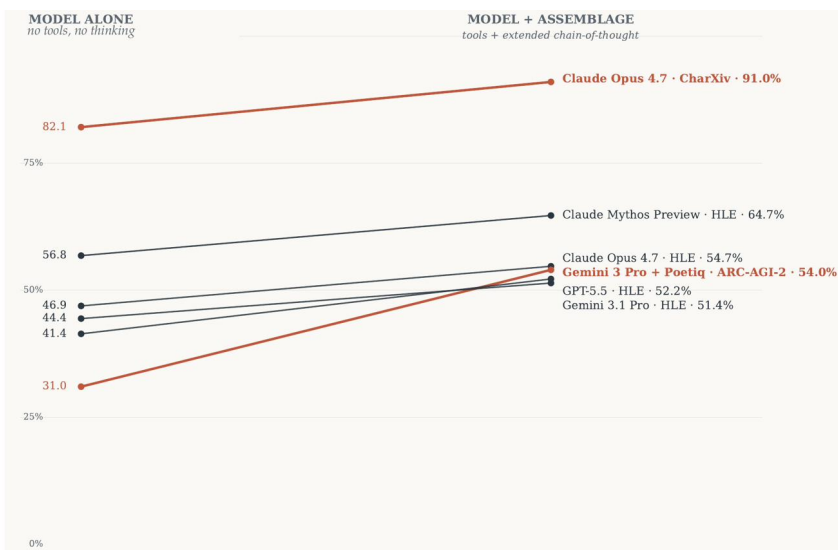


Fig. 1. The May 2026 externalization premium. Six model-benchmark pairs, each evaluated twice. The slope is what the assemblage adds. Anthropic flags that the Mythos HLE number “could indicate some level of memorization” at low effort. Sources: Opus 4.7 system card (Apr 16, 2026); Mythos Preview system card (Apr 7, 2026); GPT-5.5 launch (Apr 23, 2026); Gemini 3.1 Pro materials; ARC Prize Foundation Technical Report 2025.

OpenAI’s GPT-5 launch stated the disclaimer plainly: “AIME results with tools should not be compared directly to the performance of models without tool access; they are an example of how effectively GPT-5 leverages available tools.” The same caveat appears verbatim on the o3/o4-mini launch page. The vendor knows the property does not compose. The boundary case is **ARC-AGI-3** (arXiv:2603.24621), released March 25, 2026, designed by François Chollet

specifically to defeat extended chain-of-thought scaffolding. It drops the agent into 135 small interactive game environments with no instructions, no goals, and no rules disclosed. Crucially, evaluation runs offline on Kaggle: no external API calls, no cloud inference during scoring. Untrained humans solve every environment. At launch the four leading frontier models, run by their own vendors, scored a combined less than one percent.

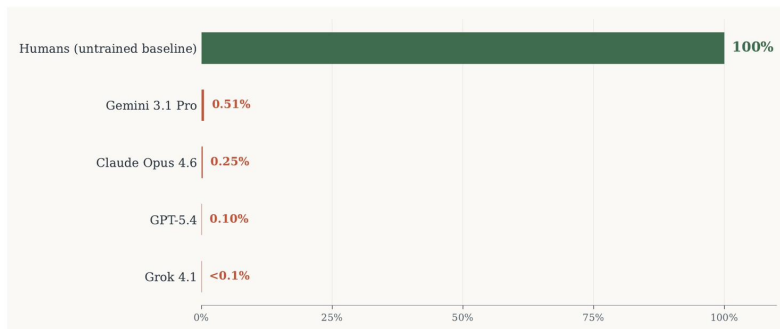


Fig. 2. When the assemblage cannot help. ARC-AGI-3 launch results, March 25, 2026. Both Opus 4.7 and GPT-5.5 shipped after the official evaluation; the public leaderboard remained below 1% across all submissions through April 30, 2026. When the benchmark defeats the very scaffolding that lifts scores everywhere else, the model alone returns nothing.

When the assemblage cannot help, the bare model has nothing to fall back on. This is the cleanest demonstration of non-compositionality the field has produced.

Composition with itself. A model computing natively can use its own outputs as inputs to its own further reasoning with the same numerical and semantic properties throughout. A delegating system round-trips through the tool layer, where every call is a serialization boundary. The training-stack analog is the trainer-versus-sampler numerics disagreement: when two components that should express the same model produce different log-probabilities for the same sequence, the gradient estimator is silently off-policy. The bug went undetected because each component was correct against its own specification.

Specification. When intelligence is internalized, the spec is the training distribution. When externalized, it is the union of the training distribution, every tool’s interface, every tool’s behavior, every parser, every retry policy, every error-handling branch, and every interaction between these. The latter is enormously larger and almost none of it is documented. The system works because the team that built it has the spec in their heads and unit tests. None of this is the model. Calling the resulting artifact “intelligent” credits one component for the achievement of an entire engineering organization.

These four are projections of the same structural fact: intelligence as we use the word is a property of *where the work happens*, and externalizing the work moves the property out of the artifact we want to claim has it.

The brittleness underneath

The tool gap measures how much the assemblage adds; perturbation measures what the model itself has: what survives when the problem is shifted by something a competent reasoner would not notice. The 2025 literature is unusually clean and unusually unkind to the standard reading of benchmark scores.

ASyMOB (17,092 problems, 35,000 mathematically-equivalent variants) reports a frontier-average drop of 77.0% $\rightarrow$ 33.4%, with 43.6 points lost to substitutions a human would not register, with a maximum single-model drop of 70.3 pp. The most advanced models in the sweep (o4-mini, Gemini 2.5 Flash) score 96.8% / 97.6% on the seed and lose only 21.7 / 21.2 points. Frontier scale buys partial robustness, not full robustness. **Putnam-AXIOM** (variable/constant renaming) drops o1-preview from 41.95% to 22.35%, a 19.6-point absolute drop, 46.8% relative; ten of the remaining 18 models show non-overlapping 95% CIs. **MSCR** (single-word adversarial) reduces OpenAI o3 by 15.88 pp on GSM8K and GPT-4o by 17.21 pp via transferred substitutions; on open-source math models, one adversarial word can erase up to 49.89% of GSM8K accuracy. **RIDE** (item-response-theory difficulty conditioning) reports a -21.73 pp average across 26 models including GPT-5 and DeepSeek-V3.1.

The most controlled result comes from a different direction. Apple’s June 2025 study ran Claude 3.7 Sonnet (Thinking), DeepSeek-R1, and o3-mini high on Tower of Hanoi. Past a complexity threshold, accuracy collapses. Lawsen’s follow-up qualified part of this: some collapse past $N \approx 10$ is a token-budget artifact, and River Crossing includes mathematically impossible instances above $N = 5$. But the more arresting finding survived independent replication. As problems near the failure threshold, the models **reduce reasoning tokens despite ample budget remaining**, as if recognizing further effort will not help. The “Rethinking the Illusion of Thinking” replication (July 2025) confirmed: *“LRMs reach peak token consumption when the task is complex but still solvable. Conversely, when the model implicitly detects that the task lies beyond its capabilities, it allocates fewer resources.”* The model gives up its budget before it fails. Not a reasoner pushing toward an answer it cannot reach, but a system that has learned to recognize the boundary of its own competence and back off, without internalizing the procedure that would solve the problem on the other

side.

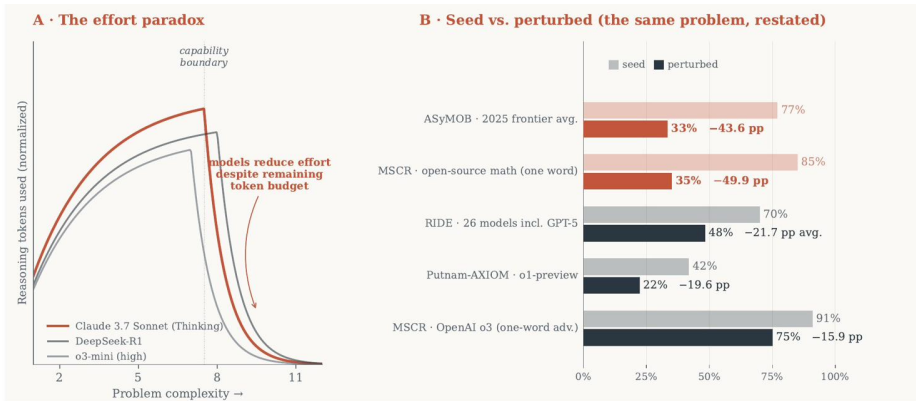


Fig. 3. What survives when the problem is shifted slightly. Panel A: three reasoning models on procedural tasks; reasoning-token usage rises with complexity then drops sharply despite remaining budget. Panel B: seed (lighter) vs. perturbed (full color) across five benchmarks; the two largest drops (MSCR open-source math, ASyMOB frontier average) highlighted.

What ties the tool gap and the perturbation collapse together is that they are projections of the same fact from opposite directions. The tool gap shows how much the assemblage adds; the perturbation collapse shows how little the bare model has. Either alone would be deniable. Together they leave the standard reading of benchmark scores indefensible: the model that scored ninety on the original cannot do thirty on a variant a competent reasoner would call identical, but it can do ninety on the variant if you let it write a Python function. The competence is somewhere. It is not where we have been pointing.

Where the work was, where it goes

A reading of the brittleness data: of course the bare model fails: capability had to be installed somehow, and “calling a tool” is just how installation manifests at inference. On this view, externalization at inference is the result of a training pipeline that itself externalized: a verifier in an RL loop, a teacher in distillation, a curated dataset for SFT. The complaint, on this reading, is just about timing.

The rejoinder has real force, because it points at the only mechanism by which the externalization argument could be defused: gradient descent. If externalization gets internalized, the property eventually lives in weights, and the assemblage at inference is a contingent matter of cost. If it does not, externalization is structural and the attribution problem is permanent.

Three families of recent results bear on whether and how internalization is happening.

The first is **RL with verifiable rewards (RLVR)**. Mukherjee et al. (2025) measured this directly across PPO, GRPO, ORPO, KTO, DPO, SimPO, PRIME, and ten model families: *“such large gains result from updating only a small subnetwork comprising just 5 percent to 30 percent of the parameters, with the rest effectively unchanged.”* RL induces parameter sparsity that emerges without sparsity-inducing regularization; the verifier’s signal is absorbed into a small, specific subnetwork; the rest of the model is left alone. Behaviorally: DeepSeek-R1 reported pure RL on a verifiable math reward took its base from 15.6% to 71.0% pass@1 on AIME 2024, 86.7% under majority voting, without SFT priming or inference scaffolding (the *Nature* update reports 77.9% for the pure-RL endpoint). By spring 2026 GPT-5.5 and Opus 4.7 reach near-saturation on AIME without tools. Mythos Preview reports 97.6% on USAMO 2026, an Olympiad-level set graded on proofs (which resist tool shortcuts) whose answers were not in any training corpus when scored. Most strikingly: on July 21, 2025, IMO President Prof. Dr. Gregor Dolinar officially confirmed an advanced version of Gemini Deep Think had earned 35/42 — gold-medal level — solving five of six IMO 2025 problems “in natural language within the 4.5-hour time limit,” no formal-language scaffold, no manual translation. A clean year-over-year improvement from AlphaProof, which had required Lean translation for IMO 2024 silver. The reasoning lives in weights.

The second is **distillation**. On-policy distillation (Lu / Thinking Machines Lab, Oct 2025) shows the student-on-its-own-rollouts variant *“can learn the RL-trained policy in approximately 7–10× fewer gradient steps, which corresponds to a compute efficiency of 50–100×”* — meaning verifier-distilled circuits can be transferred at roughly 1–2% of the RL cost. But the move is not free. Apple’s **Distillation Scaling Laws** (Busbridge et al., arXiv:2502.08606) report a sharp regime split: distillation only beats SFT when teachers already exist or when many students share one teacher; *“distillation does not improve upon standard fully supervised learning for LLMs given sufficient model size and compute budget on the student.”* And Kim et al. (arXiv:2603.24472, March 2026), *“Why Does Self-Distillation (Sometimes) Degrade the Reasoning Capability of LLMs?”*, document drops *“of up to 40%”* on out-of-distribution math when self-distillation suppresses epistemic verbalization — internalization can corrupt as well as compress.

The third is **mechanistic interpretability**. Anthropic’s circuit-tracing on Claude 3.5 Haiku (March 2025) found two-digit addition is performed by two parallel pathways: one computing a rough approximation, the other determining the last digit precisely. The result is in weights at the circuit level. But the same paper documents an unsettling corollary: *“Claude seems to be*

unaware of the sophisticated ‘mental math’ strategies that it learned during training. If you ask how it figured out that $36+59$ is 95, it describes the standard algorithm involving carrying the 1.” The verbal account is not faithful to the computation. This cuts both ways: it confirms internalization at the circuit level, but it also confirms that the chain-of-thought we point at as evidence of “reasoning” can be confabulation grafted over an unrelated internal computation. The post-hoc explanation is itself a form of externalization: produced for an external interpreter (the user) whose relationship to the model’s internal computation is contingent.

So the rejoinder has a sharply bounded form. **Externalization can be internalized: for bounded operations, by RL against a verifier, given enough compute and a clean reward signal.** This works for two-digit arithmetic, modular addition (Nanda et al. fully reverse-engineered the Fourier-circuit algorithm a transformer learns for $a + b \bmod p$), competition mathematics, and code under unit-test verification. It does not work for unbounded compositional reasoning, novel game environments, or distribution-shifted procedural execution. The brittleness data are exactly what it looks like when the verifier covered some operations and not others: operations the verifier saw are robust; operations one step outside its distribution collapse. The externalization argument is not a claim about an eternal limit; it is a claim about the present moment. Right now, in May 2026, the structural pattern holds: the work that constitutes the intelligence we are claiming happens in the toolchain at least as often as in weights.

There is one further reason the rejoinder cannot fully neutralize the argument, which has to do with bounded versus unbounded computation. The expressivity literature gives this an exact form. Let $\text{TF}_d^{\log}$ denote depth- d log-precision decoder-only transformers and let $\text{TF}_d^{\log}[t(n)]$ denote the same with $t(n)$ chain-of-thought decoding steps. Then under standard assumptions:

$$\begin{aligned} \text{TF}_d^{\log} &\subseteq \text{TC}^0, & \text{TF}_d^{\log}[O(\log n)] &\subseteq \text{TC}^0, \\ \text{TF}_d^{\log}[\Theta(n)] &= \text{L}, & \text{TF}_d^{\log}[\text{poly}(n)] &= \text{P}. \end{aligned} \tag{2}$$

A fixed-depth transformer without scratchpad cannot decide directed-graph connectivity, cannot simulate arbitrary automata, cannot solve linear equalities (Merrill & Sabharwal, ICLR 2024). With a linear number of intermediate generation steps it can simulate automata; with a polynomial number, Turing machines for that many steps. Length, not content, extends expressive power. Even padding tokens with no informational content extend computational depth (Merrill & Sabharwal, NeurIPS 2025; Pfau, Merrill & Bowman, ICLR 2025). This is the formal version of “the bare model literally cannot do certain things; the scratchpad does computational work.”

But the depth can equally well live inside the model via recurrent depth iteration. Geiping et al.’s recurrent-depth model (NeurIPS 2025) shows a 3.5B-parameter network “*can improve its performance on reasoning benchmarks, sometimes dramatically, up to a computation load equivalent to 50 billion parameters*”, by iterating its hidden state at test time. Internal depth and external scratchpad are theoretically interchangeable in the relevant sense. The question is not whether intelligence requires depth (it does) but whether the depth lives *inside the artifact whose intelligence we are claiming*. The TC^0 argument supports internalization-as-possible; the brittleness data argue it is not yet what is happening at frontier scale.

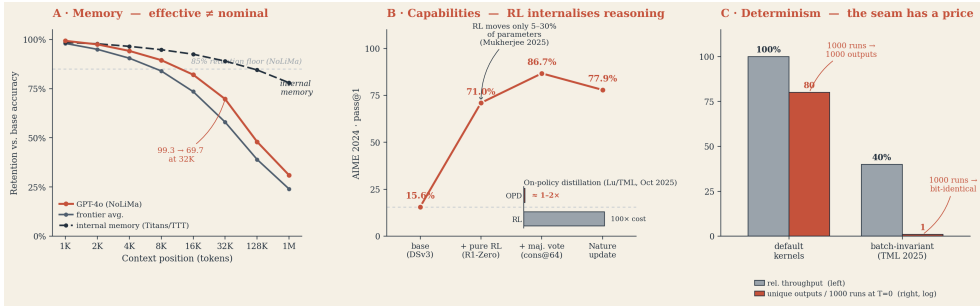


Fig. 4. Three axes along which the externalization line is currently being moved. A Memory: the gap between nominal and effective context length. NoLiMa (Modarressi et al., ICML 2025) reports GPT-4o falling from 99.3% to 69.7% by 32K tokens once literal-match cues are removed; frontier average degrades faster. Internal-memory architectures (Titans/TTT/Huginn) hold retention longer because the work happens in the activations, not in the prompt. **B Capabilities:** DeepSeek-R1’s pure-RL trajectory on AIME 2024 (15.6 → 71.0% pass@1, 86.7% with majority voting; Nature update 77.9%), with Mukherjee’s finding that RL across PPO/GRPO/DPO/etc. moves only 5–30% of parameters and the inset showing on-policy distillation reaching the same endpoint at ≈1–2% of RL compute (Lu/TML, Oct 2025). **C Determinism:** bit-identical inference is achievable but costs roughly 60% of throughput; the same kernel swap (TML 2025) collapses 1000 unique outputs at $T = 0$ to 1.

What “computing natively” actually requires

The right bar: the system’s latent state must be the locus of the relevant work. Not its outputs — outputs are cheap and any orchestration can produce correct outputs. The work has to happen in the activations. Whether with explicit reasoning tokens, a hidden scratchpad, or pure latent-space reasoning is a second-order architectural choice. The first-order requirement is that the work happen somewhere inside the system whose intelligence we are claiming.

Not a counsel of perfection. No problem with using high-throughput

external storage for unbounded knowledge (no model can fit the web in its weights). No problem with deferring genuinely irreducible operations (cryptographic verification, real-time sensor reads, communication with other agents) to dedicated systems. The objection is to systematic externalization of operations that constitute the intelligence we are claiming. Arithmetic is one. State maintenance over a long task is one. Stepwise reasoning under verification is one. If those leak into the toolchain, the model is not the thing being praised when the system performs well.

A research program recently active asks where capability has to live. On a casual reading it looks like architectural maximalism; the real move is austere. Build a system whose latent state simultaneously plays the role of compute, memory, and I/O. A maturity target is set by four conditions that must hold simultaneously: **Turing completeness**; **universal programmability** (capabilities installable by inputs and callable later); **behavioral consistency** under ordinary use; and **machine-native semantics** (primitives developed by the system rather than borrowed from a host).

Concretely: a system that solves any problem the human-plus-tools assemblage can solve, given equivalent compute, without externalizing the constitutive operations. A system you can program by handing it examples that it converts into circuitry it then calls. A system that produces the same answer at temperature zero across runs, replicas, and load conditions. A system whose explanations of itself are mechanistically faithful to its computations.

This is a high bar. Frontier production systems do not meet it: their compute is fixed-depth without scratchpad extension, their memory is short-context plus retrieval, their I/O is the tool layer. None of this is wrong as engineering. It is wrong as the answer to “is the model intelligent?” because it changes what “the model” refers to mid-sentence. The four-condition test is also what makes the externalization argument falsifiable: ship a system that passes it and behaves as well as the assemblage-based stacks, and the argument loses force.

An architecture sketch: the Closed-Loop Architecture

It helps to make the bar concrete. What follows is a single architecture, called the **Closed-Loop Architecture (CLA)**, that by construction satisfies the four-condition test without externalizing the constitutive operations. The point is not that any production system today implements this; none does at frontier scale. The point is to show that the bar is specifiable. The pieces exist in the literature; they have not yet been composed into a single artifact, and frontier systems compose them with the tool-call layer instead.

Definition

Let $x_{1:T}$ be the input. The CLA is the tuple $(\mathcal{P}, \mathcal{R}, \mathcal{C}, \mathcal{M}, \rho)$ with:

1. Prelude $\mathcal{P} : \mathbb{R}^{|x| \times v} \rightarrow \mathbb{R}^{T \times d}$, a shallow encoder mapping tokens to per-position embeddings $e_{1:T} = \mathcal{P}(x_{1:T})$.
2. Recurrent block $\mathcal{R}_\theta : \mathbb{R}^{T \times d} \times \mathbb{R}^{T \times d} \rightarrow \mathbb{R}^{T \times d}$, a parameter-shared block iterated $r(x)$ times (defined below). Inside $\mathcal{R}$, attention uses a non-diagonal, input-dependent state-transition update (§4) so that the per-token recurrence can recognize regular languages natively, not in token-space but in latent-space.
3. Coda $\mathcal{C} : \mathbb{R}^{T \times d} \rightarrow \mathbb{R}^{T \times v}$, a shallow decoder.
4. Fast-weight memory $\mathcal{M}_t \in \mathbb{R}^{d_M \times d_M}$, an in-pass learned associative memory, updated by the rule (§5) below. $\mathcal{M}$ lives inside the forward pass; it is not a KV-cache offered to the user.
5. Recurrence policy $\rho : \mathbb{R}^d \rightarrow [1, R_{\max}(n)]$, a per-token learned policy choosing how many recurrence steps each position receives, with budget at least $R_{\max}(n) = \Omega(\log n)$ to clear the relevant complexity class (Merrill & Sabharwal, 2025).

The closed loop

The forward pass is the fixed point of an iterated map. With $s^{(0)} \sim \mathcal{N}(0, \sigma^2 I)$, for each position $i \in [T]$:

$$s_i^{(k+1)} = \mathcal{R}_\theta(e_i \oplus s_i^{(k)}; \mathcal{M}_t, S_t), \quad k = 0, 1, \dots, \rho(e_i) - 1, \quad (3)$$

where $\oplus$ denotes channel-wise concatenation, S_t is the recurrent attention state defined below, and the output is $\hat{y} = \mathcal{C}(s^{(\rho(e))})$. The loop closes inside the activation tensor: each iteration’s output is the next iteration’s input, with no token serialization in between. This is the architectural translation of “the work has to happen in the activations.”

Component 1: non-diagonal state transition (escaping TC^0 at the layer level)

Let $u_t \in \mathbb{R}^d$ be the input to a recurrent attention head at step t , and let $S_t \in \mathbb{R}^{d_k \times d_v}$ be its state. We use a generalized delta rule (RWKV-7 style) with input-dependent, non-diagonal transition:

$$S_t = (\text{diag}(w_t) - \hat{\kappa}_t \hat{\kappa}_t^\top a_t) S_{t-1} + v_t \hat{\kappa}_t^\top, \quad (4)$$

where $\hat{\kappa}_t = \kappa_t / \|\kappa_t\|$, and $(w_t, \kappa_t, v_t, a_t)$ are linear projections of u_t . The crucial property: the transition matrix $A_t = \text{diag}(w_t) - \hat{\kappa}_t \hat{\kappa}_t^\top a_t$ has eigenvalues outside the diagonal-only spectrum that confines diagonal SSMs (Mamba, S4) to TC^0 at finite precision (Merrill, Petty & Sabharwal, ICML 2024, “Illusion

of State”). With (4) the per-token recurrence provably recognizes all regular languages natively, without buying expressivity from a polynomial-length scratch in the token stream.

Component 2: fast-weight memory (closing the long-tail loop)

The memory $\mathcal{M}_t$ is itself a small parametric model whose weights are updated within the forward pass by a Titans-style surprise-gated rule. Let (k_t, v_t) be projections of the recurrent state. Define “surprise” as the gradient of an associative reconstruction loss with respect to the current memory:

$$m_t = \beta_t m_{t-1} - \nabla_{\mathcal{M}} \ell(\mathcal{M}_{t-1}; k_t, v_t), \quad \mathcal{M}_t = (1 - \alpha_t) \mathcal{M}_{t-1} + \eta_t m_t, \quad (5)$$

with input-dependent gates: α_t (forgetting), β_t (momentum), η_t (learning rate). The associative loss is $\ell(\mathcal{M}; k, v) = \frac{1}{2} \|\mathcal{M}k - v\|^2$ in the linear case, or a deeper MLP variant. The update happens at each token, inside the forward pass, with no KV-cache exposed to the orchestrator. “Memory” here means: a small network whose weights change with the input the network is currently processing.

Component 3: adaptive recurrence depth (work where work is needed)

Tokens that demand multi-step composition get more iterations; tokens that don’t, don’t. The Mixture-of-R recursions policy ρ is trained jointly, with a budget regularizer:

$$\mathcal{L}_\rho = \mathcal{L}_{\text{task}} + \lambda \cdot \mathbb{E}_{i,x}[\rho(e_i)], \quad (6)$$

which selects the smallest depth that suffices per token. This is the architectural form of “think harder when you need to,” implemented in the layer count rather than in token count.

Component 4: single-stack numerics (closing the seam)

Training and inference share one numerical pipeline: batch-invariant kernels for matmul / RMSNorm / attention reductions (Thinking Machines Lab, 2025), unified FP8 for both the trainer forward pass and the rollout engine, and identical attention-mask conventions on both sides. Concretely, this enforces $\pi_\theta = \pi_{\text{train}} = \pi_{\text{roll}}$ at the per-token level, eliminating the need for the truncated-IS correction (1). The system is on-policy by construction, not by patch.

Component 5: verifier-distilled training, verifier-free inference

RL with verifiable rewards (DeepSeek-R1-style) is used during training only; the verifier is removed from the inference path. Mukherjee et al. (2025) provide the empirical handle: across PPO/GRPO/DPO/etc., 5–30% of parameters

move under RL, and the moving subnetwork is consistent across seeds. The verifier signal is therefore localizable: after training, what the verifier taught is circuitry, not call sites.

The four conditions, checked

Turing completeness. With $R_{\max}(n) = \Omega(n)$ and unbounded scratch in the latent state (the per-iteration $s^{(k)}$ tensor), the looped architecture simulates Turing machines for $\text{poly}(n)$ steps (Giannou et al., ICML 2023; Merrill & Sabharwal, NeurIPS 2025). The scratch is the system’s own activations, not a token sequence handed to a sandbox.

Universal programmability. The fast-weight memory $\mathcal{M}_t$ is provably equivalent to a step of online gradient descent on an associative regression problem (Sun et al., ICML 2025; von Oswald et al., ICML 2023, “in-context learning is gradient descent”). Programs handed in via context become, mechanically, weights the system writes to a small in-pass network and calls later. “Programmability” is achieved without a tool registry.

Behavioral consistency. Single-stack numerics enforce bit-identical inference at $T = 0$ and bit-identical π_θ between rollout and update. The system has a determinable behavior, not an emergent one.

Machine-native semantics. The verifier-distilled circuits live in the 5–30% subnetwork that RL moves; mechanistic interpretability tools (sparse autoencoders, attribution graphs) operate on these weights, not on a tool-call log. The semantics are the system’s, not the host’s.

What CLA does not do

It is not the web. Long-tail factual recall still wants high-throughput external storage; CLA keeps that; it just does not call the database the model. It is not a sandbox. Cryptographic verification, real-time sensor I/O, and inter-agent communication still go to systems designed for them. The objection is to systematic externalization of operations that constitute the intelligence we are claiming: arithmetic, state maintenance, stepwise reasoning, length-extrapolating composition. Those go inside.

Where the literature already is

Recent research is converging from two directions that need to be kept apart.

On the internalization side of the line. Test-time-learning architectures collapse the controller-versus-memory distinction: Sun et al.’s TTT-layers (ICML 2025) make “*the hidden state a machine learning model itself, and the update rule a step of self-supervised learning*” — exactly (5) with linear $\mathcal{M}$ — and “*can*

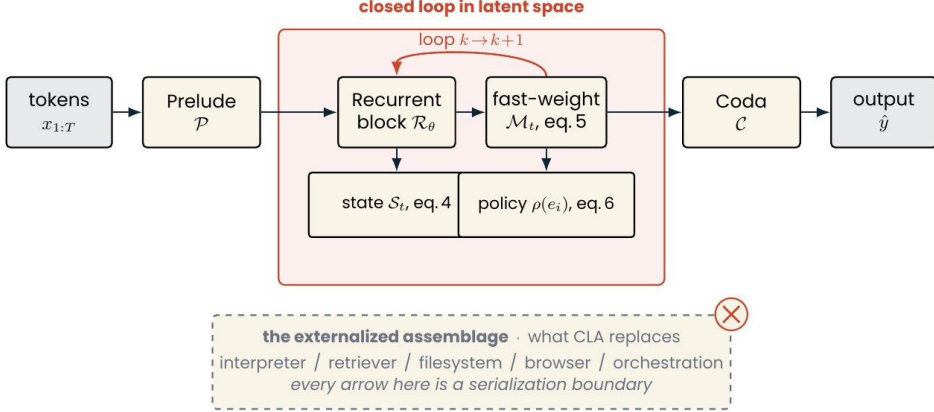


Fig. 5. The Closed-Loop Architecture. Five components: prelude $\mathcal{P}$, recurrent block $\mathcal{R}_\theta$ with non-diagonal state transition S_t , in-pass fast-weight memory $\mathcal{M}_t$, per-token depth policy ρ , and coda $\mathcal{C}$. The loop closes in the activation tensor: iteration $k \rightarrow k+1$ does not pass through the token stream. Below: the externalized assemblage that CLA replaces; every arrow that crosses out of the latent state is a serialization boundary.

keep reducing perplexity by conditioning on more tokens, while Mamba cannot after 16k context.” Behrouz et al.’s Titans (Dec 2024) extends this to long-term neural memory with surprise-driven update gates and outperforms much larger models on the BABILong reasoning benchmark; the MIRAS follow-up (Behrouz, Razaviyayn & Mirrokni, 2025) recasts the design space as four components — architecture, attentional bias, retention gate, algorithm. RWKV-7 (Peng et al., March 2025) deploys (4) at 7B scale and proves regular-language recognition in Appendix D, the cleanest published architectural break of the TC^0 ceiling. Mixture-of-R recursions (Bae et al., NeurIPS 2025) implements (6) with per-token adaptive depth and reports Pareto-better perplexity at matched FLOPs with $2\times$ inference throughput. Geiping et al.’s Huginn-3.5B (NeurIPS 2025) trains the recurrent-depth template at 800B tokens and achieves “computation load equivalent to 50 billion parameters” by iterating its hidden state at test time. None of these systems composes all five CLA components simultaneously, and none has been pretrained at frontier scale; the architecture sketch is a specification of what such a composition would have to look like.

On the externalization side. The dominant production paradigm is what Letta (the MemGPT successor) explicitly calls “*LLM-as-an-Operating-System*”: the LLM is like the CPU, and its context window is like the RAM, serving as the model’s working memory. Memory is external by design, accessed through tool calls (`memory_replace`, `memory_insert`, `archival_memory_search`). The

benchmark situation is illustrative: Mem0 (Chhikara et al., arXiv:2504.19413) reports a 26% relative win over OpenAI on LoCoMo at $< 7K$ tokens per retrieval; the original Zep paper claimed 84% on the same benchmark; Mem0’s reproduction corrected this to $58.44 \pm 0.20\%$; Zep’s counter-claim is 75.14%. Three numbers, one benchmark, the same harness ostensibly running. The point is not which framework is best — it is that a 25-point swing across vendor harnesses for the same memory operation is itself the externalization argument. Capability is not located at any one of the components.

The naming has caught up with the structure. By mid-2025 the field had renamed prompt engineering as **context engineering** — Karpathy, June 2025: *“the delicate art and science of filling the context window with just the right information for the next step”*; Tobi Lütke (Shopify), early summer 2025: *“the art of providing all the context for the task to be plausibly solvable by the LLM”*; Gartner, October 2025: *“context engineering is in, and prompt engineering is out.”* The renaming is itself an admission. We have stopped pretending the model is doing the work alone.

The same gap shows up in long-context evaluation. **NoLiMa** (Modarressi et al., ICML 2025) defines “effective length” as the longest context where a model maintains $\geq 85\%$ of its base score and finds that *“even GPT-4o, one of the top-performing exceptions, experiences a reduction from an almost-perfect baseline of 99.3% to 69.7% [at 32K].”* At 32K tokens, eleven of the evaluated models drop below 50% of their short-context performance. Chroma’s **Context Rot** study (Hong et al., July 2025) extends this across 18 frontier models including GPT-4.1, Claude Opus 4, Gemini 2.5, and Qwen3: *“even under these minimal conditions, model performance degrades as input length increases, often in surprising and non-uniform ways.”* Effective context length is roughly 65–70% of nominal across the frontier as of mid-2026. The scaling curve in Fig. 4A is what this looks like as a structural fact: doubling the advertised window does not double the usable one, and the difference is exactly the externalization premium re-priced.

The latent-reasoning rebuttal has a sharper version worth pre-empting. Continuous-thought architectures (Coconut, Hao et al., Dec 2024) feed the model’s hidden state back as the next embedding rather than decoding to words, on the hypothesis that reasoning can occur in continuous representational space below the token level. Modest gains on small benchmarks. But a December 2025 causal/adversarial analysis (“Do Latent Tokens Think?”) found *“latent tokens in COCONUT showed minimal sensitivity to perturbations and displayed a clustered embedding pattern, further confirming that these tokens act as placeholders rather than meaningful representations of reasoning.”* The architecture is not yet a substitute for verbalized chain-of-thought at frontier scale: depth lives somewhere, but probably not in the latent tokens themselves, which is

exactly the case for putting depth in (3) instead, where it is causally active by construction.

Verifiable-reward training applied beyond mathematics is the second front: theorem proving with proof checkers, code with unit tests, robotic manipulation with privileged-state physics simulators — domains where the verifier is an oracle at training and absent at inference. The DeepSeek-R1 trajectory (15.6 $\rightarrow$ 71.0% AIME via pure RL on a verifiable reward, with 5–30% of parameters moved per Mukherjee) is the proof of concept for the “verifier removed at inference” clause of CLA.

The honest cost

Frontier prototypes that try to internalize symbolic computation in a single architecture currently report something like four percent accuracy on two-digit arithmetic, with reprompting (system-level conditioning) lifting this to the eighties, consistent with theoretical bounds on what a fixed-depth transformer can compute without scratchpad. But where the operation is reasoning rather than raw arithmetic, internalization has come further than pessimistic accounts allow. RL on a fixed pre-trained base with verifiable rewards and no learned reward proxy climbed from 15.6% to 71.0% pass@1 on AIME 2024, 86.7% with majority voting. The same paradigm with a proof checker reached IMO silver in 2024 and gold in 2025 with end-to-end natural-language reasoning. By spring 2026 GPT-5.5 and Opus 4.7 reach near-saturation on AIME without tools; Mythos hits 97.6% on USAMO 2026. None of these are orchestration wins.

CLA-style architectures pay in three currencies. The fast-weight update (5) requires a gradient step inside the forward pass; current TTT-Linear / TTT-MLP implementations run at $\sim 5\%$ of peak FLOPS on H100s versus $\sim 50\%$ for vanilla attention (“Test-Time Training Done Right,” arXiv:2505.23884). Recurrent depth (3) costs a multiplicative factor in latency proportional to $\mathbb{E}[\rho(e)]$. Single-stack numerics (Component 4) constrains kernel choice and forecloses some of the throughput optimizations production stacks rely on — concretely, the $\sim 60\%$ throughput cost of batch-invariant kernels in Fig. 4C. These are real costs. They are also costs of specifying what the system is, which is the sole charge of the four-condition test.

Externalization continues to win in production: it ships, it composes with existing infrastructure, it is what every frontier agent is built on. But the question is not “what works in production”; it is “what are we claiming when we talk about intelligence.” Conflating the two is the move I’m refusing. Imagine a chess engine that, asked for the best move, dispatches the position

to a far stronger engine over an RPC and returns that engine’s answer. The wrapper is correct; by any benchmark it plays grandmaster-level chess. We would not, on this evidence, say the wrapper plays grandmaster-level chess. We would say it is good at calling the external engine. To know how good it is at chess, you would disable the tool layer and watch what happened.

The current state of frontier AI is, in many evaluations, closer to this analogy than is comfortable. Not in the trivial sense that models cannot function without tools, but in the deep sense that the reasoning, persistence, and calculation we point at when we say “intelligent” are happening, in the systems we deploy, on the far side of the tool boundary.

Moving the work back inside

The proposal: **stop accepting the tool call as load-bearing. Move the work back into the system whose intelligence you are claiming.** A unified architecture is years away; the engineering challenges are real. But the philosophical move is available right now, and it is what gives the term “intelligent system” any specifiable referent at all. Apply the test consistently. When evaluating a model, evaluate the model, not the assemblage. When reporting capability gains, report them at the layer where the capability lives. When claiming frontier systems can reason, ask whether the reasoning happened in weights or middleware. When drawing the boundary around “the AI” for legal, ethical, or scientific purposes, draw it where the intelligence actually is.

The same attribution error is observable in ourselves. METR’s mid-2025 randomized controlled trial (Becker, Rush et al., arXiv:2507.09089) gave half of 16 experienced open-source developers access to early-2025 frontier AI tooling and half not. The developers expected a 24% speedup; afterwards they self-reported 20%; the actual measured effect was a **19% slowdown** (95% CI [+2%, +39%]). The assemblage of human-plus-IDE-plus-Cursor-plus-Claude was less productive than the human alone, and the human could not tell. METR’s February 2026 follow-up cautioned “*the data from our new experiment gives us an unreliable signal of the current productivity effect of AI tools*” but reported, on returning developers, an estimated –18% speedup with CI [–38%, +9%]; the only RCT remains the original. The intelligence the assemblage delivered did not become the developer’s intelligence; the assemblage’s productivity did not become the developer’s productivity; the perceptual error ran in the same direction. The discipline applies to both sides of the keyboard.

A kind of intellectual defeatism says these distinctions don’t matter: the assemblage is what we have, the interface is the system. The most thoughtful version draws on the extended-mind: humans externalize too (calculators,

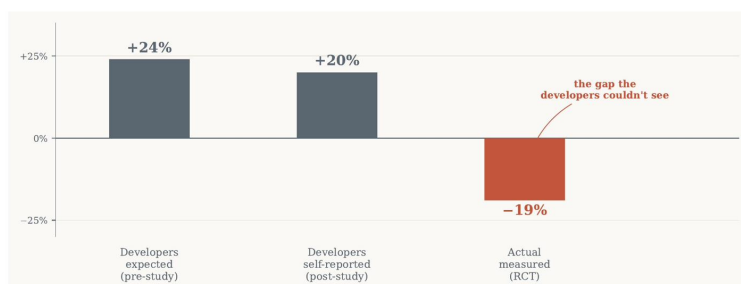


Fig. 6. What developers thought the AI was doing for them, and what it was. METR RCT, July 2025: 16 experienced open-source developers with ≥ 5 years on the codebase. METR’s February 2026 follow-up notes 2026 productivity is likely higher than the 2025 estimate but explicitly cautions the new signal is unreliable. The directional finding stands. Source: *arXiv 2507.09089*; METR blog 2026-02-24.

notebooks, language itself) and we are still intelligent. Why hold language models to a stricter standard? The answer is we don’t. The standard is the same; the LLM case violates it in a way notebook-mediated cognition does not. Andy Clark himself, after his original parity argument with Chalmers, narrowed the criteria for an external process to count as cognitive: it must be *constantly available, automatically endorsed, easily accessible, and previously endorsed*. A model whose intelligence is attributed because of an external Python interpreter the user could substitute or remove — configurable at the system-prompt level, swappable between providers, sometimes present and sometimes not — fails the constancy criterion. The 2025 *Extended Hollowed Mind* critique sharpens this: “*the very capacity that leads generative AI to be celebrated as a powerful ‘cognitive extender’ is precisely what allows users to bypass the mental effort required to build a resilient internal architecture for deep reasoning.*” The extended-mind defense is weaker for the LLM case than for the notebook case, not stronger.

The boundary between model and toolchain is not an implementation detail; it is the line that decides what we are talking about. The model is what gets shipped, sold, benchmarked, and called “GPT-5.5” or “Claude Opus 4.7.” The system card describes the model. The leaderboard scores the model. The pricing is per-token of the model. The category mistake is what happens when you draw the boundary at the assemblage for prediction but at the model for attribution.

A system that cannot compute without delegating to a filesystem is not a computing system; it is an interface to one. A model that cannot reason without delegating the reasoning is not a reasoning model; it is a router for one. These can be useful artifacts. They are not what we have been calling them.

The non-compositionality of intelligence is, in the end, a claim about discipline. If a property does not compose, you are not allowed to attribute it to the composition. You must attribute it to wherever it actually lives, or admit it lives nowhere in particular and the assemblage is not the thing your word refers to. The words we use to describe our systems should refer to the systems we are describing. Right now, far too often, they don't. The fix begins with noticing.

References

Compositionality, nondeterminism, and the seam

He, H. *Defeating Nondeterminism in LLM Inference*. Thinking Machines Lab, Sep 10, 2025.

Yao, Y. et al. *On the Rollout-Training Mismatch in Modern RL Systems*. openml.org/papers/2025/paper116, 2025.

Fu, Y. et al. *Your Efficient RL Framework Secretly Brings You Off-Policy*. FlashRL, Aug 2025.

ROLL Flash. [arXiv:2510.11345](https://arxiv.org/abs/2510.11345), October 2025.

Frontier model assemblage premium (Figure 1)

Anthropic. *Claude Opus 4.7 System Card*. April 16, 2026.

Anthropic. *Claude Mythos Preview System Card*. April 7, 2026.

OpenAI. *Introducing GPT-5.5*. April 23, 2026.

Google. *Gemini 3.1 Pro launch materials*. April 2026.

ARC Prize Foundation. *2025 Technical Report*.

OpenAI. *Introducing GPT-5*. August 2025.

ARC-AGI-3 (Figure 2)

ARC Prize Foundation. *ARC-AGI-3: A New Challenge for Frontier Agentic Intelligence*. [arXiv:2603.24621](https://arxiv.org/abs/2603.24621), March 27, 2026.

Perturbation collapse (Figure 3)

Shalyt, M. et al. *ASyMOB: Algebraic-Symbolic Mathematics Benchmark*. NeurIPS 2025. [arXiv:2505.23851](https://arxiv.org/abs/2505.23851).

Gulati, A. et al. *Putnam-AXIOM: A Functional and Static Benchmark*. ICML 2025. [arXiv:2508.08292](https://arxiv.org/abs/2508.08292).

Yao, Y. et al. *MSCR: Minimal Single-Character Replacements*. [arXiv:2511.08055](https://arxiv.org/abs/2511.08055), Nov 2025.

Mirzadeh, I. et al. *GSM-Symbolic / GSM-NoOp*. ICLR 2025. [arXiv:2410.05229](https://arxiv.org/abs/2410.05229).

Shojaee, P. et al. *The Illusion of Thinking*. Apple ML Research. [arXiv:2506.06941](https://arxiv.org/abs/2506.06941), June 2025.

Lawsen, A. *Comment on the Illusion of Thinking*. arXiv:2506.09250, June 2025.
Liang, J. et al. *Rethinking the Illusion of Thinking*. arXiv:2507.01231, July 2025.
RIDE: Item-Response-Theory Difficulty Conditioning. arXiv:2511.04120, Feb 2026.

Internalization at the circuit level

Mukherjee, S. et al. *Reinforcement Learning Finetunes Small Subnetworks in Large Language Models*. arXiv:2505.11711, May 2025.
DeepSeek-AI. *DeepSeek-R1: Incentivizing Reasoning Capability in LLMs via RL*. arXiv:2501.12948, Jan 2025; *Nature* s41586-025-09422-z, 2025.
Google DeepMind. *Advanced version of Gemini with Deep Think achieves gold-medal standard at the IMO*. July 21, 2025.
Lindsey, J. et al. *On the Biology of a Large Language Model*. Anthropic, transformer-circuits.pub, March 2025.
Nanda, N. et al. *Progress measures for grokking via mechanistic interpretability*. arXiv:2301.05217, 2023.

Scaling laws for post-training (Figure 4B)

Lu, K. et al. *On-Policy Distillation*. Thinking Machines Lab, Oct 27, 2025.
Busbridge, D. et al. *Distillation Scaling Laws*. arXiv:2502.08606, Feb 2025; updated July 2025.
Kim, J. et al. *Why Does Self-Distillation (Sometimes) Degrade the Reasoning Capability of LLMs?* arXiv:2603.24472, March 25, 2026.
Brown, W. *Why the SFT–RL pipeline works, where on-policy distillation fits, and how self-distillation goes wrong*. April 2026.

Memory and context scaling (Figure 4A)

Behrouz, A., Zhong, P. & Mirrokni, V. *Titans: Learning to Memorize at Test Time*. arXiv:2501.00663, Dec 2024.
Behrouz, A., Razaviyayn, M. & Mirrokni, V. *Titans + MIRAS: Helping AI have long-term memory*. Google Research, 2025.
Sun, Y. et al. *Learning to (Learn at Test Time): RNNs with Expressive Hidden States*. ICML 2025. arXiv:2407.04620.
Test-Time Training Done Right. arXiv:2505.23884, May 2025.
Geiping, J. et al. *Scaling up Test-Time Compute with Latent Reasoning: A Recurrent Depth Approach*. NeurIPS 2025. arXiv:2502.05171.
Modarressi, A. et al. *NoLiMa: Long-Context Evaluation Beyond Literal Matching*. ICML 2025. arXiv:2502.05167.
Hong, K., Troynikov, A. & Huber, J. *Context Rot: How Increasing Input Tokens Impacts LLM Performance*. Chroma Research, July 2025.
Packer, C. et al. *MemGPT: Towards LLMs as Operating Systems*. arXiv:2310.08560.
Chhikara, P. et al. *Mem0: Building Production-Ready AI Agents with Scalable Long-Term Memory*. arXiv:2504.19413, 2025.
Karpathy, A. *Context engineering*, X / June 2025; Gartner, October 2025.

Theoretical bounds and architecture (§An architecture sketch, equations 2–6)

Merrill, W. & Sabharwal, A. *The Expressive Power of Transformers with Chain of Thought*. ICLR 2024. arXiv:2310.07923.

Merrill, W. & Sabharwal, A. *Exact Expressive Power of Transformers with Padding*. NeurIPS 2025. arXiv:2505.18948.

Pfau, J., Merrill, W. & Bowman, S. *Let’s Think Dot by Dot*. ICLR 2025. arXiv:2404.15758.

London, A. & Kanade, V. *Pause Tokens Strictly Increase the Expressivity of Constant-Depth Transformers*. NeurIPS 2025. arXiv:2505.21024.

Merrill, W., Petty, J. & Sabharwal, A. *The Illusion of State in State-Space Models*. ICML 2024. arXiv:2404.08819.

Peng, B. et al. *RWKV-7 “Goose” with Expressive Dynamic State Evolution*. arXiv:2503.14456, March 2025.

Bae, S. et al. *Mixture-of-Recursions*. NeurIPS 2025. arXiv:2507.10524.

Giannou, A. et al. *Looped Transformers as Programmable Computers*. ICML 2023. arXiv:2301.13196.

von Oswald, J. et al. *Transformers learn in-context by gradient descent*. ICML 2023.

Hao, S. et al. *Coconut: Training LLMs to Reason in a Continuous Latent Space*. arXiv:2412.06769, Dec 2024.

Zhang et al. *Do Latent Tokens Think? A Causal and Adversarial Analysis of Chain-of-Continuous-Thought*. arXiv:2512.21711, Dec 2025.

METR developer productivity (Figure 5)

Becker, J., Rush, N. et al. *Measuring the Impact of Early-2025 AI on Experienced Open-Source Developer Productivity*. arXiv:2507.09089, July 2025.

METR. *Updating our understanding of the productivity effect of AI on experienced developers*. metr.org, Feb 24, 2026.

Adjacent works referenced in the body

Clark, A. & Chalmers, D. *The Extended Mind*. Analysis 58 (1998), 7–19.

Clark, A. *Supersizing the Mind*. Oxford University Press, 2008.

The Extended Hollowed Mind. 2025.

Ryle, G. *The Concept of Mind*. Hutchinson, 1949.

[Editor’s Note: Arguments are mine, writing is Claude’s. This is partially an experiment in getting Claude to help speed-write and structure technical research blogs, drafted as artifacts and refined via “debate”. I have too many blog ideas I never get around to writing up; the models finally feel good enough to help out (hopefully; let me know what you think).]